# Tema 1: Introducción a Data Streams y arquitecturas comunes.

Ángel Miguel García Vico
Rafael Pastor Vargas

amgarcia@scc.uned.es
rpastor@scc.uned.es

**Módulo 3: Gestión de datos en tiempo real**

**Infraestructuras Computaciones para Procesamiento de Datos Masivos**

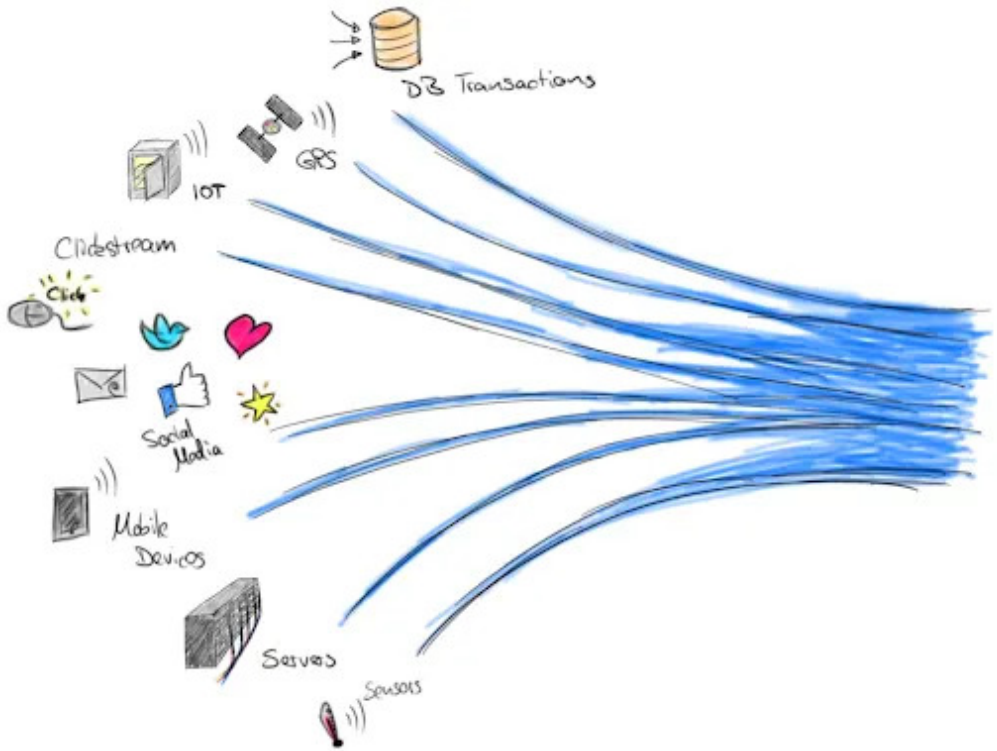
30 de noviembre de 2022



# Flujos de datos: Motivación

Vivimos rodeados de datos. La mayoría de ellos son **generados automáticamente** a partir de nuestro entorno y comportamiento y utilizados en exclusiva entre las máquinas:

- Comunicación **muy rápida**.
- Estos datos **contienen conocimiento** muy relevante que puede ser aprovechado (y que normalmente no se procesa).
- Datos efímeros: validez muy corta.



# Flujos de datos: Motivacion

- Muchas aplicaciones reales funcionan en entornos continuos.
- Se requiere incorporar constantemente nuevos datos al modelo y que éste se actualice de manera **incremental**.
- Algunos algoritmos permiten este funcionamiento incremental: kNN o Naïve Bayes.
- Pero la mayoría no permiten ser actualizados incrementalmente de manera sencilla sin realizar cambios significativos en el modelo.
- Además, la distribución subyacente en los datos puede cambiar a lo largo del tiempo, haciendo que el modelo aprendido con datos pasados no sea válido.

# Flujos de datos

- Un flujo de datos $S = \{x_1, x_2, \ldots, x_t, \ldots\}$ se define como una **secuencia ordenada** y **potencialmente infinita** de datos que llega al sistema **a lo largo del tiempo** a una **velocidad que puede ser variable.**
- Principales características:
  - Secuencia ordenada: un dato es generado antes que otro y llega antes al sistema. Este orden se puede dar por *timestamp* u orden de llegada. **¡Puede implicar dependencia de datos!**
  - Infinita: el flujo puede estar funcionando de manera indefinida, almacenarlo por completo implica **memoria infinita**.
  - A lo largo del tiempo: implica que tenemos una **disponibilidad parcial** del conjunto de datos en un instante de tiempo $t$. Además, implica que la distribución subyacente que genera los datos **puede variar** a lo largo del tiempo.
  - Velocidad variable: el tiempo de procesamiento de un dato es limitado, habitualmente se asume una llegada rápida de datos, que implica un **procesamiento rápido** $\rightarrow$ ¿respuestas aproximadas?.

# Ejemplo: Cálculo del máximo

Para ejemplificar la dificultad, vamos a ver un ejemplo simple del cálculo del valor máximo (o mínimo) de un flujo de datos. En un conjunto de datos tradicional es trivial, pero en flujos de datos puede haber diferentes opciones:

- ¿El máximo a lo largo de todo el flujo? ¿Y si cambia el dominio de la variable en el tiempo?
- ¿El máximo durante un período de tiempo? ¿Cómo elegimos dicho perído? ¿Un período fijo en tiempo, instancias, o un período variable?

# Flujos de datos: diferencias con minería de datos tradicional

Minería de datos:

- Tamaño finito.
- Disponibilidad de datos total.
- Procesamiento/almacenamiento puede ser lento.
- Distribución de datos idéntica.
- Sin actualización de modelo.
- Datos etiquetados al momento.

Flujos de datos:

- Tamaño infinito.
- Disponibilidad de datos parcial.
- Procesamiento/almacenamiento rápido.
- Distribución variable.
- Modelo incremental.
- *Feedback* retardado.

A todo esto hay que sumarle: **cantidad masiva de información** $\rightarrow$ Procesamiento distribuido (Big *Streaming* Data).

# Flujos de datos: casos de uso

- Detección de anomalías o malfuncionamientos.
- Casos de fraude en banca online.
- Sistemas de recomendación.
- Predicción de clicks en una web.
- Detección de tendencias emergentes.
- etc.

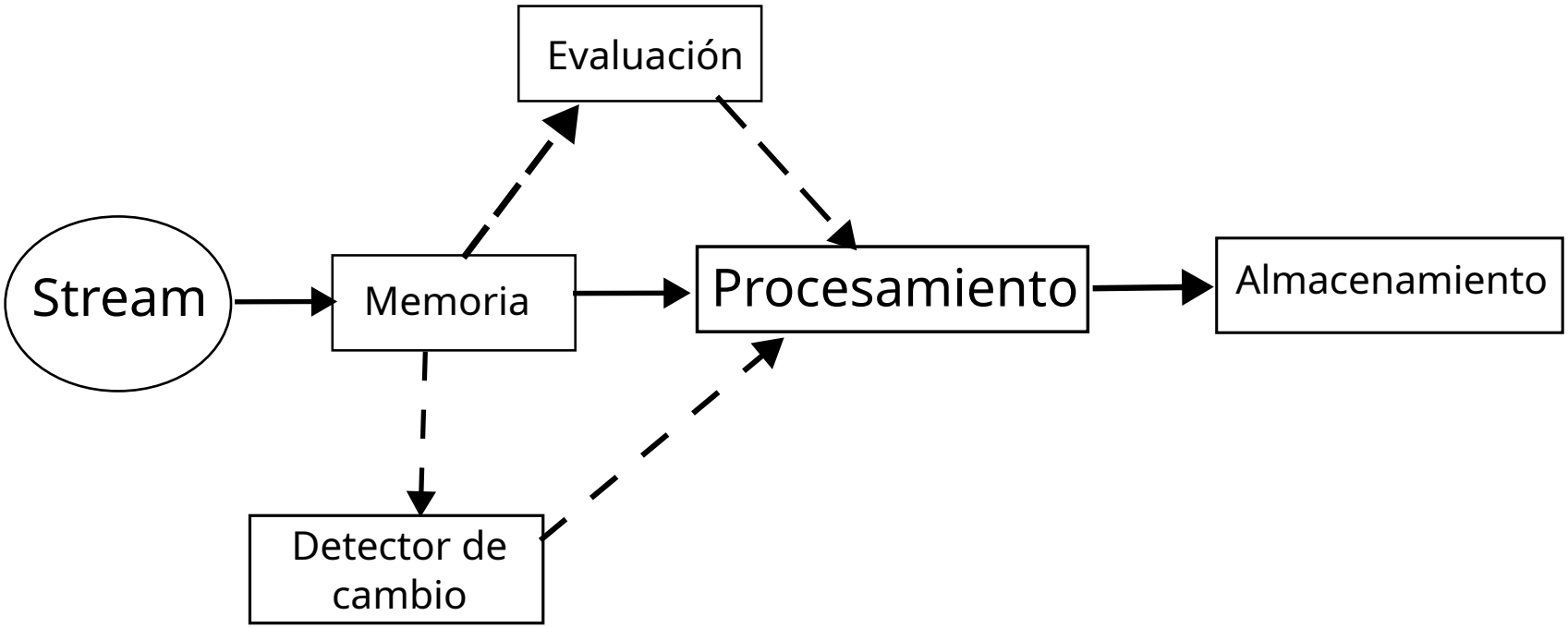
# Flujos de datos: elementos

Según las características de los flujos de datos se hace necesario tener en consideración cuatro elementos fundamentales cuando se abordar un análisis de flujos de datos:

1. Memoria: Actúa como un *buffer* de los datos provenientes del stream. Almacena datos durante un período $t_{alm}$ de tiempo. Puede descartar directamente algunos datos que considere innecesarios.
2. Algoritmo de aprendizaje/procesamiento. Se encarga de extraer el conocimiento en sí y debe actualizarse y adaptarse de acuerdo a las características del flujo. Idealmente, el tiempo de procesamiento $t_{proc}$ debe ser menor a $t_{alm}$: $t_{proc} \leq t_{alm}$.
3. Monitorización del cambio. Es un sistema externo, auxiliar al algoritmo de aprendizaje, que intenta detectar posibles cambios en la naturaleza de los datos y avisar al algoritmo de aprendizaje de estos cambios para que se tomen las acciones oportunas.
4. Evaluación del rendimiento. Se necesitan enfoques que no requieran todos los datos.

# Flujos de datos: elementos

Conexión entre los diferentes elementos. Las flechas con línea discontinua indican elementos opcionales.



# Flujos de datos: elementos

En esta asignatura nos centraremos principalmente en la memoria, es decir, cómo se recopilan datos desde diferentes fuentes al sistema, con sistemas como Apache Kafka, y en la infraestructura de procesamiento distribuido de Apache Spark Streaming, pero veremos brevemente todos los elementos en este tema. Para más información sobre algoritmos y métodos de ciencia de datos en flujos de datos, pueden consultar el libro:

João Gama. **Knowledge Discovery from Data Streams**. Chapman y Hall/CRC, 2010

# Memoria en flujo de datos

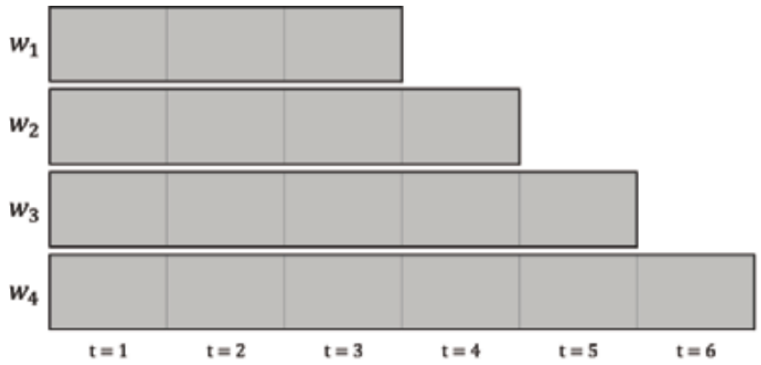
Hemos visto que, idealmente, un método de flujo de datos debería ser capaz de procesar uno a uno los datos en cuanto llegan al sistema. Sin embargo, debido a limitaciones de procesamiento, esto no suele ser habitual. Asimismo, existe el problema del cambio de concepto: es interesante mantener información de los **datos recientes**, olvidando los antiguos. Para ello hay varias estrategias que pueden unirse entre si:

- Mantener estadísticos básicos: media, desviación, etc.
- Muestreo de datos para reducir la cantidad a procesar.
- Histogramas y sinopsis para describir la distribución estadística de los datos.
- Ventanas de datos o *windowing*. Apache Spark Streaming utiliza esta técnica.

# Memoria en flujo de datos: Ventanas de datos

Una ventana de datos $W_t \subset S$ es un subconjunto del stream $S$ que contiene únicamente aquellas instancias más recientes o interesantes. Existen diferentes definiciones de ventana:

- *Landmarks*: Se establecen diferentes instantes, sucesos o hitos $h_i$ en el que a partir de dicho punto, las instancias que llegan al sistema se acumulan en una ventana hasta la llegada del siguiente hito $h_{i+1}$, en el que se vacía la memoria. Un hito puede ser, por ejemplo, la detección de un cambio de concepto. Es útil para trabajar con datos que pertenecen al mismo concepto, pero se corre el riesgo de saturar la memoria e incrementar significativamente el tiempo de cómputo si la ventana crece demasiado. En Apache Spark Streaming son llamadas "Sesiones": el hito en este caso es un período de inactividad.

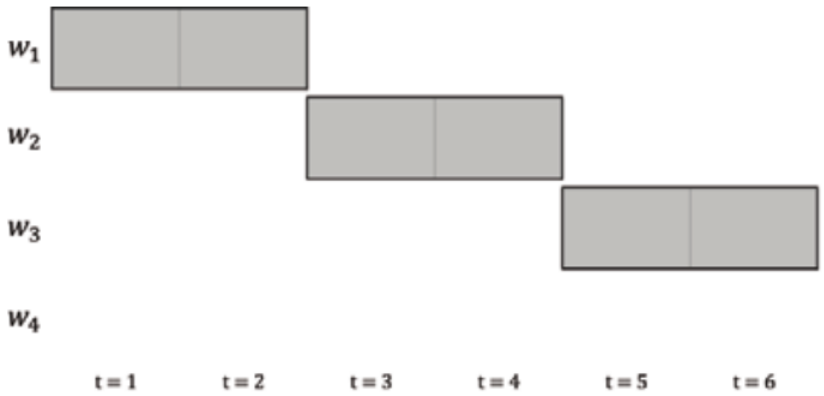


# Memoria en flujo de datos: Ventanas de datos

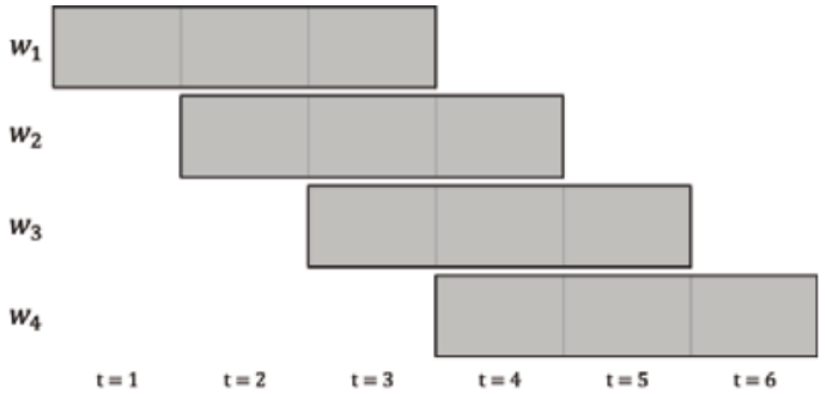
- *Sliding windows*: Mantienen únicamente los datos más recientes y descartan los más antiguos conformen van llegando al sistema:
  - Son fáciles de implementar. Es una cola FIFO.
  - Habitualmente son de **tamaño fijo**. Pero pueden ser también de tamaño variable (ADWIN).
  - Dos ventanas consecutivas pueden presentar solapamiento entre sí, depende del número de instancias que se introduzcan en la cola.
  - Las ventanas pueden establecerse por dos criterios: número de instancias (sequence-based) o duración en el tiempo (timestamp-based), consistente en incluir aquellas instancias que llegan dentro del período de tiempo establecido.

# Memoria en flujo de datos: Ventanas de datos

- En Apache Spark Streaming, los flujos de datos se representan con la clase `DStream` que, por defecto, son ventanas *timestamp-based* sin solapamiento. También se pueden crear con solapamiento. Cada una de estas ventanas temporales es representada, a su vez como un `RDD`. Es decir, Spark recopilarán todos los datos que lleguen dentro de un período de $t$, creará un RDD (con las transformaciones previas necesarias) y se procesará. Este proceso se repite indefinidamente.




Sin solapamiento.




Con solapamiento.

# Evaluación de modelos de ciencia de datos

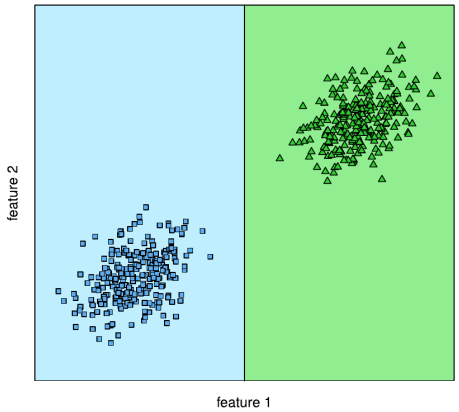
En el desarrollo de algoritmos (de ciencia de datos o no), es necesario medir el rendimiento del sistema. No obstante, a veces para calcular algunas métricas de rendimiento es necesario disponer del conjunto de datos completo. Por ejemplo, para realizar cross-validation. Esto no se puede realizar en flujo de datos, ¿cómo se puede hacer?

- Una posible solución es el enfoque de evaluación *prequential* o *test-then-train*: brevemente, el modelo o algoritmo se evalúa en primer lugar con la ventana de datos entrante y, a continuación, dichos datos se usan para entrenar el algoritmo.

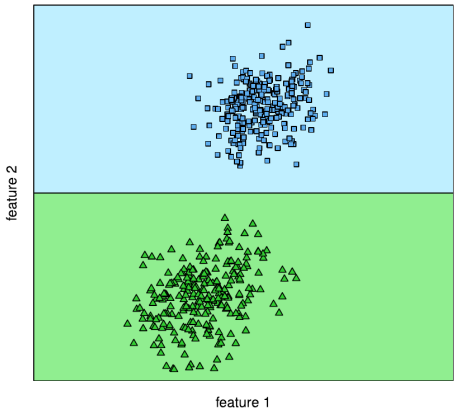
Joao Gama, Raquel Sebastiao y Pedro Pereira Rodrigues. «On evaluating stream learning algorithms». En: **Machine learning** 90.3 (2013), págs. 317-346

# Cambio de concepto

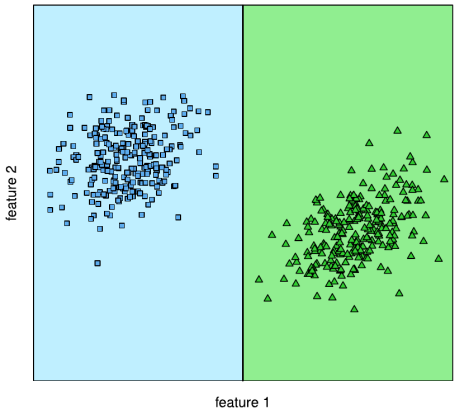
Habitualmente, la distribución de los datos no es estacionaria, es decir, que evoluciona a lo largo del tiempo. El hecho de que se produzca este cambio se denomina **cambio de concepto**.




(a) Initial distribution.




(b) Real concept drift.

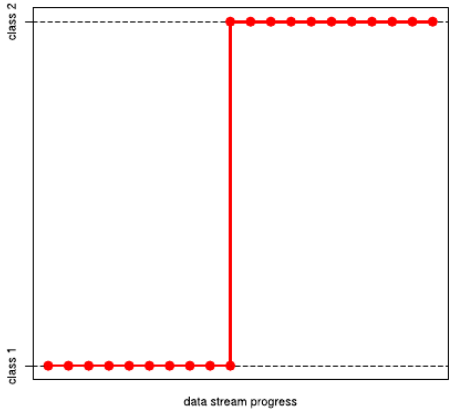



(c) Virtual concept drift.

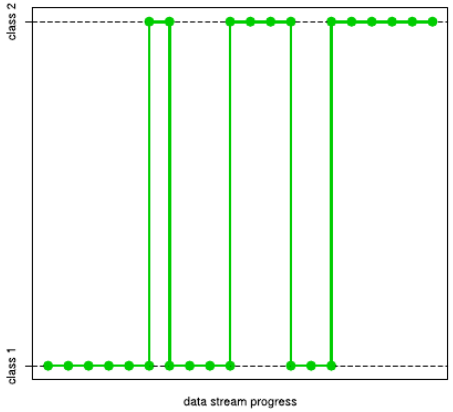
Esto implica que los algoritmos entrenados en un instante de tiempo pueden no ser válidos en instantes posteriores

# Cambio de concepto

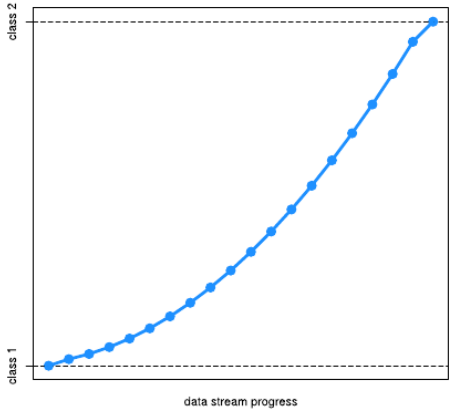
El cambio visto anteriormente se puede producir a lo largo del tiempo de diferente forma:



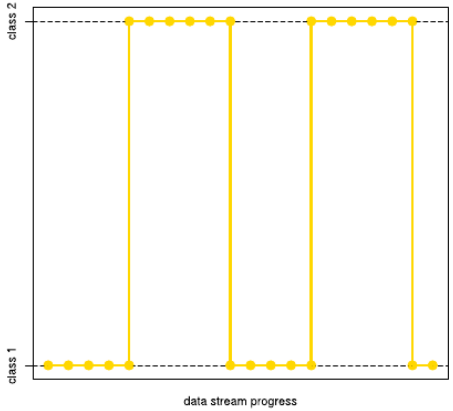

(a) Sudden.



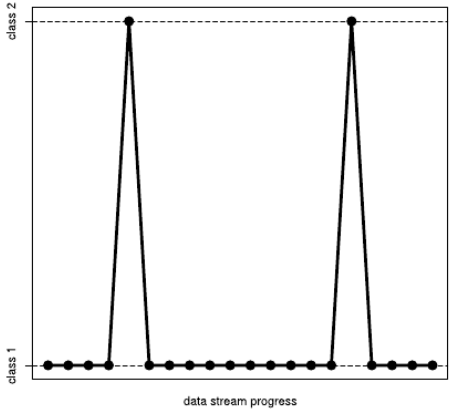

(b) Gradual.



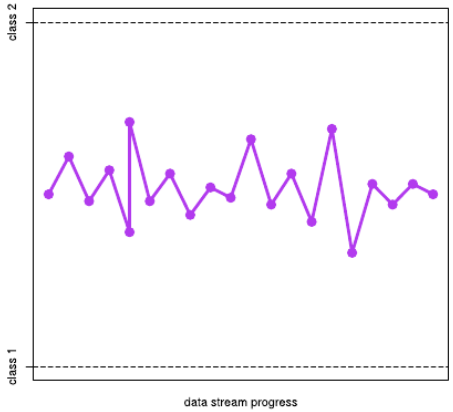

(c) Incremental.




(d) Recurring.



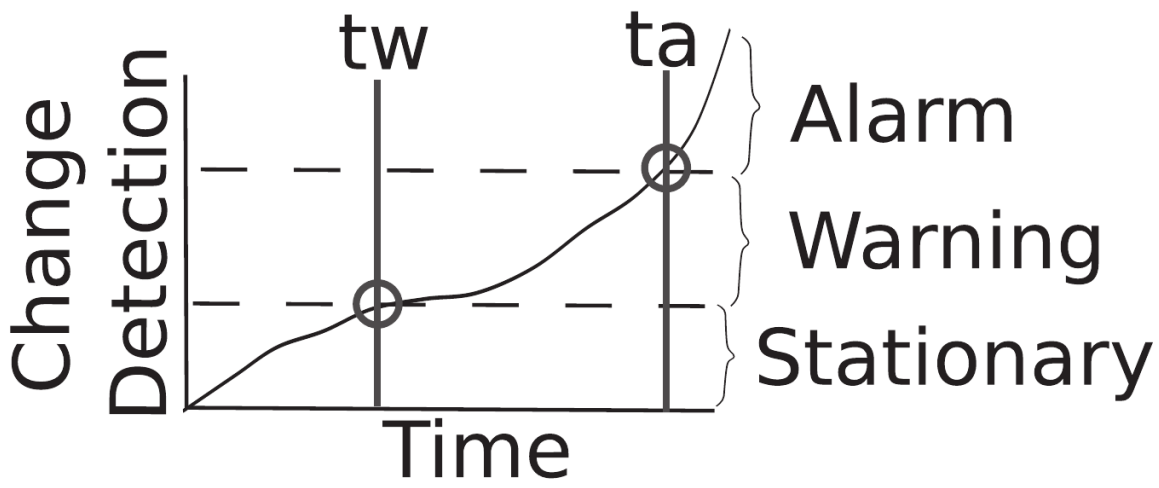

(e) Blips.




(f) Noise.

# Cambio de concepto

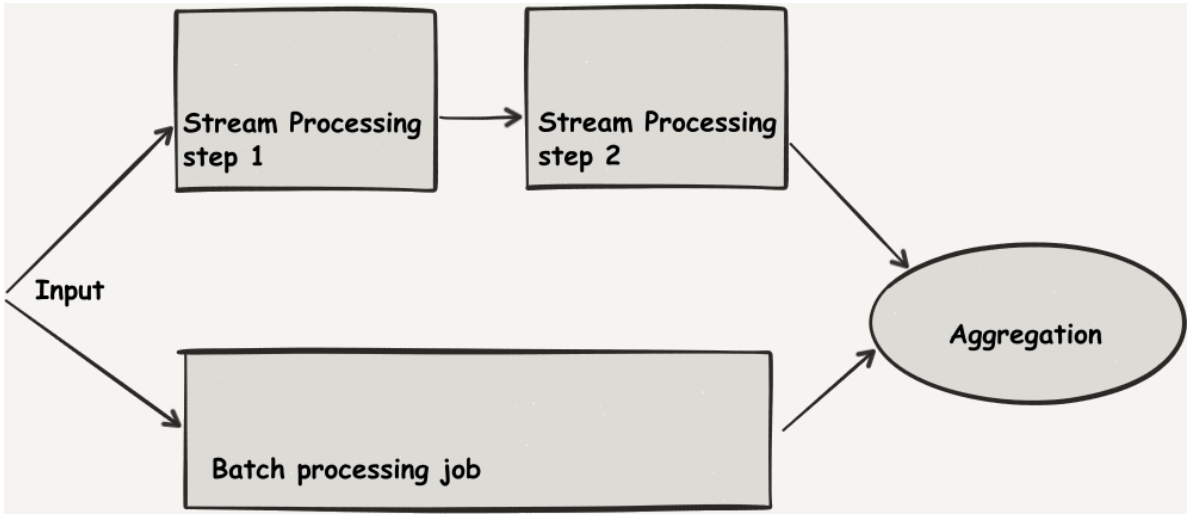
Habitualmente se emplea un sistema de monitorización del cambio de concepto en el que se siguen unas métricas objetivo: el error del modelo, la desviación de los datos respecto la media actual, etc.




Se establecen dos niveles: uno de alerta y otro de alarma. En el nivel de alerta se crea una ventana auxiliar que almacena estos datos en caso de producirse la alarma de cambio de concepto.

# Arquitecturas de streaming

- Arquitectura Lambda. Tiene dos partes: una *online* y otra *offline*
  - Fase online: aplica un procesamiento ligero de streaming a los datos actuales.
  - Fase offline: aplica un procesamiento más costoso a datos acumulados, siguiendo un proceso más tradicional.




Ejemplo: Algoritmo de IA.

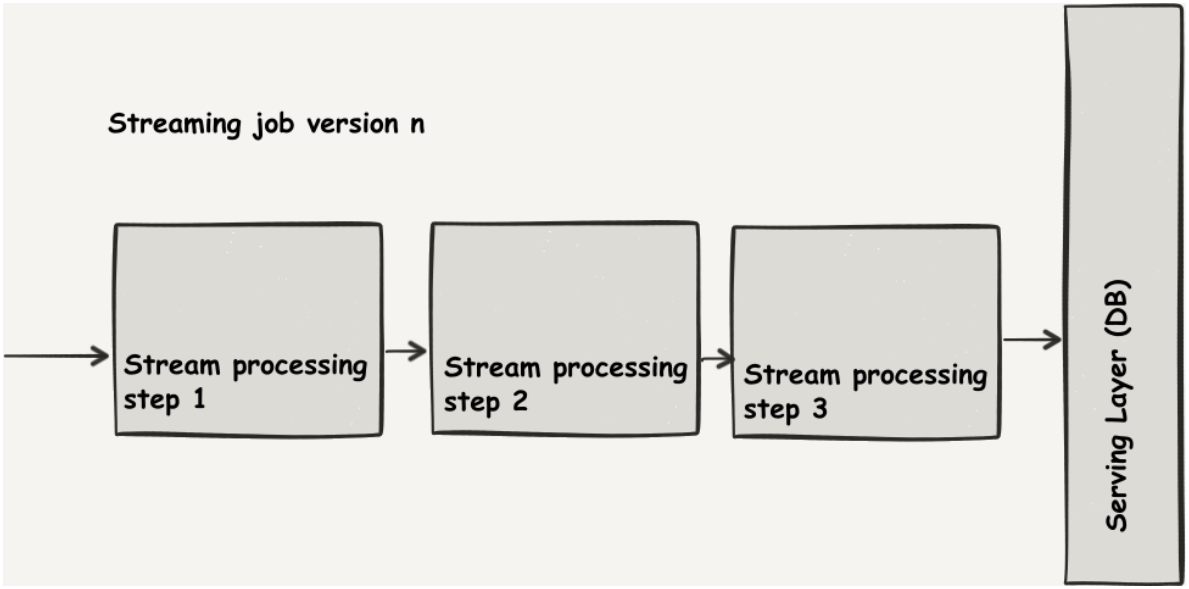
- Fase online: inferencia.
- Fase offline: entrenamiento.

# Arquitecturas de streaming

- Arquitectura Lambda. Tiene dos partes: una *online* y otra *offline*
  - Combina procesamiento batch y streaming.
  - Se duplica funcionalidad.
  - Más universal (casi todo puede funcionar con esta arquitectura)
  - Pros:
    - Ré-calculo de datos y correción de errores sencilla.
    - Capacidad de procesas grandes volumenes de datos
  - Cons:
    - Hay dos sistemas: más complejo.
    - A veces no es necesaria esta arquitectura.

# Arquitecturas de streaming

- Arquitectura Kappa. Sólo tiene fase *online* y todas las fases de procesamiento aplican procesos de data stream.




Ejemplo: Algoritmo de IA incremental.

# Arquitecturas de streaming

Arquitectura Kappa:

- Más rápido.
- Optimiza mejor los recursos.
- Pros:
  - Permite re-calcular datos y corregir errores.
  - Implementación única
- Cons:
  - No todos los problemas pueden representarse como streaming.
  - Restricciones de streaming limita mucho el sistema.

# Lectura Recomendada


- João Gama. **Knowledge Discovery from Data Streams**. Chapman y Hall/CRC, 2010. Capítulo 1.
- Tyler Akidau, Slava Chernyak y Reuven Lax. **Streaming systems: the what, where, when, and how of large-scale data processing**. .ºReilly Media, Inc.", 2018. Capítulo 1.
- Gerard Maas y Francois Garillot. **Stream Processing with Apache Spark: Mastering Structured Streaming and Spark Streaming**. O'Reilly Media, 2019. Capítulo 3.
- Joao Gama, Raquel Sebastiao y Pedro Pereira Rodrigues. «On evaluating stream learning algorithms». En: **Machine learning** 90.3 (2013), págs. 317-346
- Imen Khamassi y col. «Discussion and review on evolving data streams and concept drift adapting». En: **Evolving Systems** 9.1 (2018), págs. 1-23
- Sergio Ramírez-Gallego y col. «A survey on data preprocessing for data stream mining: Current status and future directions». En: **Neurocomputing** 239 (2017), págs. 39-57